```
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_BMP280.h>
#include <Adafruit_SHT31.h>
#include <U8g2lib.h>
#include <ThingSpeak.h>
#include <Preferences.h>
#include "soc/gpio_reg.h"
#include "soc/soc.h"

// Wi-Fi settings
const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";
WiFiClient client;
Preferences preferences;

// Environmental readings channel
unsigned long channelID = 3493634;
const char* writeAPIKey = "YOUR_PRIMARY_THINGSPEAK_WRITE_KEY";

// Playing-session channel
unsigned long usageChannelID = 3552297;
const char* usageWriteAPIKey = "YOUR_USAGE_THINGSPEAK_WRITE_KEY";

// Score channel
unsigned long scoreChannelID = 3558530;
const char* scoreWriteAPIKey = "YOUR_SCORE_THINGSPEAK_WRITE_KEY";
```

```
// Sensors
Adafruit_BMP280 bmp;
Adafruit_SHT31 sht31 = Adafruit_SHT31();

// OLED setup
U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(
  U8G2_R0, U8X8_PIN_NONE
);

const unsigned long DISPLAY_MS = 3000;

// Thermistor and Kalman filter
const int NODE_A_PIN = 35;
const float NODE_A_DIVIDER = (100.0 + 47.0) / 47.0;

const int MUX_ADC_PIN = 34;
const float PIN6_DIV = (100.0 + 47.0) / 47.0;

const float THERM_CAL_SLOPE = 0.111f;
const float THERM_CAL_INTERCEPT = -0.59f;

float thermKalmanEstimate = NAN;
float thermKalmanVariance = 1.0;
const float THERM_KALMAN_Q = 0.01;
const float THERM_KALMAN_R = 0.12;
```

```
float kalman(float z) {
  if (isnan(thermKalmanEstimate)) {
    thermKalmanEstimate = z;
    return thermKalmanEstimate;
  }

  thermKalmanVariance += THERM_KALMAN_Q;
  float kalmanGain =
    thermKalmanVariance /
    (thermKalmanVariance + THERM_KALMAN_R);

  thermKalmanEstimate +=
    kalmanGain * (z - thermKalmanEstimate);

  thermKalmanVariance *= (1.0f - kalmanGain);

  return thermKalmanEstimate;
}

float thermRaw = NAN;
float thermKF = NAN;

// Guitar condition score
const float W_HUM = 0.60f;
const float W_TEMP = 0.30f;
const float W_PLAYTIME = 0.08f;
const float W_PRESSURE = 0.02f;
```

```
const float TEMP_AGREE_C = 2.0f;
const float PLAYTIME_GOAL_MIN = 60.0f;
const float PRESSURE_TOLERANCE_HPA = 20.0f;

float humScore = NAN;
float tempScore = NAN;
float playtimeScore = NAN;
float pressureScore = NAN;
float guitarScore = NAN;
float scoreTempC = NAN;
float pressureBaselineHpa = NAN;

bool tempDisagree = false;
bool lastTempDisagree = false;

float computeTieredScore(
  float val,
  float idealLow,
  float idealHigh,
  float okayLow,
  float okayHigh,
  float maxPenaltyRange
) {
  if (val >= idealLow && val <= idealHigh) {
    return 100.0f;
  }
```

```
  if ((val >= okayLow && val < idealLow) ||
      (val > idealHigh && val <= okayHigh)) {
    return 70.0f;
  }

  float dist = 0.0f;

  if (val < okayLow) {
    dist = okayLow - val;
  } else {
    dist = val - okayHigh;
  }

  float score = 40.0f -
                (dist / maxPenaltyRange) * 40.0f;

  return (score < 0.0f) ? 0.0f : score;
}

void computeGuitarScore(
  float thermC,
  float bmpC,
  float shtC,
  float rh,
  float pressureHpa,
  float totalPlaytimeMinutes
```

```
) {
  bool thOk = !isnan(thermC);
  bool bmpOk = !isnan(bmpC);

  tempDisagree = false;

  if (thOk && bmpOk) {
    if (fabsf(thermC - bmpC) <= TEMP_AGREE_C) {
      scoreTempC = (thermC + bmpC) / 2.0f;
    } else {
      tempDisagree = true;
      scoreTempC = shtC;
    }
  } else {
    tempDisagree = true;
    scoreTempC = shtC;
  }

  if (isnan(scoreTempC) || isnan(rh)) {
    humScore = tempScore = guitarScore = NAN;
    return;
  }

  float tempF = scoreTempC * 9.0f / 5.0f + 32.0f;

  humScore = computeTieredScore(
    rh, 45.0f, 55.0f, 40.0f, 60.0f, 15.0f
```

```
);

tempScore = computeTieredScore(
  tempF, 70.0f, 75.0f, 65.0f, 80.0f, 15.0f
);

// Score reaches 100 at the configured cumulative playtime goal.
playtimeScore = constrain(
  100.0f * totalPlaytimeMinutes / PLAYTIME_GOAL_MIN,
  0.0f,
  100.0f
);

// Compare pressure with the first valid reading saved for this device.
if (!isnan(pressureHpa)) {
  if (isnan(pressureBaselineHpa)) {
    pressureBaselineHpa = pressureHpa;
    preferences.putFloat("pBase", pressureBaselineHpa);
  }

  pressureScore = constrain(
    100.0f * (1.0f -
      fabsf(pressureHpa - pressureBaselineHpa) /
      PRESSURE_TOLERANCE_HPA),
    0.0f,
    100.0f
  );
```

```
  } else {
    pressureScore = 0.0f;
  }

  guitarScore =
      W_HUM * humScore
    + W_TEMP * tempScore
    + W_PLAYTIME * playtimeScore
    + W_PRESSURE * pressureScore;
}

const char* scoreLabel(float s) {
  if (s >= 80.0f) return "SUITABLE";
  if (s >= 50.0f) return "ATTENTION";
  return "UNSUITABLE";
}

// CD4051 and piezo
const int PIN_MUX_SEL = 27;

const int PIEZO_SAMPLES = 120;
const int PIEZO_SAMPLE_INTERVAL_US = 100;

const int PIEZO_MIN_TAP_MV = 195;
const int PIEZO_BASELINE_MARGIN_MV = 30;
const int PIEZO_REARM_MARGIN_MV = 20;
```

```
const float PIEZO_BASELINE_ALPHA = 0.025f;

// Smooth piezo burst readings for comparison.
// Tap detection still uses raw Vpp.
const float PIEZO_KF_Q = 100.0f;
const float PIEZO_KF_R = 400.0f;
float piezoKx = NAN;
float piezoKP = 1.0f;
float piezoFilteredMv = NAN;

const unsigned long PIEZO_REFRACTORY_MS = 140;
const int PIEZO_REARM_QUIET_SAMPLES = 2;

unsigned long lastPiezoTapMs = 0;

float piezoPeakMv = 0;
float piezoNoiseFloorMv = NAN;
float piezoThresholdMv = PIEZO_MIN_TAP_MV;

bool piezoArmed = true;

int piezoQuietSamples = 0;
int tapCount = 0;

// Capacitive touch sensor
const int PIN_DRIVE = 25;
const int PIN_SENSE = 26;
```

```
bool sleepMode = false;

// Multiplexer functions
void muxSelect(int ch) {
  digitalWrite(PIN_MUX_SEL, ch);

  delayMicroseconds(1000);

  analogReadMilliVolts(MUX_ADC_PIN);
  analogReadMilliVolts(MUX_ADC_PIN);
}

float readV(int pin, float div) {
  long s = 0;

  for (int i = 0; i < 32; i++) {
    s += analogReadMilliVolts(pin);
  }

  return (s / 32.0) / 1000.0 * div;
}

float readThermV() {
  muxSelect(0);
  return readV(MUX_ADC_PIN, PIN6_DIV);
}
```

```
// Playing-session tracking
unsigned long firstSessionTapMs = 0;
unsigned long lastSessionTapMs = 0;

int sessionTapCount = 0;

unsigned long completedSessions = 0;
uint64_t cumulativePlayingMs = 0;

bool sessionHasTap = false;
bool previousSleepMode = false;

void finishPlayingSession() {
  if (!sessionHasTap || sessionTapCount == 0) {
    Serial.println("# No playing session to save.");

    sessionHasTap = false;
    sessionTapCount = 0;
    firstSessionTapMs = 0;
    lastSessionTapMs = 0;

    return;
  }

  unsigned long durationMs =
    lastSessionTapMs - firstSessionTapMs;
```

```
float durationSeconds = durationMs / 1000.0f;
float tapRate = 0.0f;

if (durationSeconds > 0.0f) {
  tapRate = sessionTapCount * 60.0f / durationSeconds;
}

uint64_t newCumulativeMs =
  cumulativePlayingMs + durationMs;

unsigned long newSessionNumber =
  completedSessions + 1;

Serial.println();
Serial.println("===== PLAYING SESSION COMPLETE =====");

Serial.printf("First tap: %lu ms\n", firstSessionTapMs);
Serial.printf("Last tap: %lu ms\n", lastSessionTapMs);
Serial.printf("Duration: %.2f seconds\n", durationSeconds);
Serial.printf("Taps: %d\n", sessionTapCount);
Serial.printf("Tap rate: %.2f taps/min\n", tapRate);
Serial.printf("Session #: %lu\n", newSessionNumber);

if (usageChannelID == 0 ||
    String(usageWriteAPIKey) ==
    "PASTE_NEW_CHANNEL_WRITE_API_KEY") {
```

```
  Serial.println(
    "# Set usageChannelID and usageWriteAPIKey before uploading session data."
  );
 } else if (WiFi.status() != WL_CONNECTED) {
  Serial.println(
    "# Session not uploaded: Wi-Fi is disconnected."
  );
 } else {
  ThingSpeak.setField(1, durationSeconds);
  ThingSpeak.setField(2, sessionTapCount);
  ThingSpeak.setField(3, tapRate);
  ThingSpeak.setField(
    4,
    (float)newCumulativeMs / 60000.0f
  );
  ThingSpeak.setField(5, (long)newSessionNumber);

  int response = ThingSpeak.writeFields(
    usageChannelID,
    usageWriteAPIKey
  );

  if (response == 200) {
    cumulativePlayingMs = newCumulativeMs;
    completedSessions = newSessionNumber;

    // Save the updated values in ESP32 storage.
```

```
      preferences.putULong64("playMs", cumulativePlayingMs);
      preferences.putULong("sessions", completedSessions);

      Serial.println(
        "# Playing session uploaded successfully."
      );
    } else {
      Serial.printf(
        "# Playing-session upload failed. Code: %d\n",
        response
      );
    }
  }

  Serial.println("=====================================");
  Serial.println();

  sessionHasTap = false;
  sessionTapCount = 0;
  firstSessionTapMs = 0;
  lastSessionTapMs = 0;
}

// Piezo sampling and tap detection
float kalmanPiezo(float measurementMv) {
  if (isnan(piezoKx)) {
    piezoKx = measurementMv;
```

```
    return piezoKx;
  }

  piezoKP += PIEZO_KF_Q;
  const float gain = piezoKP / (piezoKP + PIEZO_KF_R);
  piezoKx += gain * (measurementMv - piezoKx);
  piezoKP *= (1.0f - gain);

  return piezoKx;
}

float piezoBurst() {
  muxSelect(1);

  int mn = 5000;
  int mx = 0;

  for (int i = 0; i < PIEZO_SAMPLES; i++) {
    int v = analogReadMilliVolts(MUX_ADC_PIN);

    if (v < mn) mn = v;
    if (v > mx) mx = v;

    delayMicroseconds(PIEZO_SAMPLE_INTERVAL_US);
  }

  const float pp = (float)(mx - mn);
```

```
piezoFilteredMv = kalmanPiezo(pp);
const unsigned long now = millis();

if (isnan(piezoNoiseFloorMv)) {
  piezoNoiseFloorMv = piezoFilteredMv;
}

piezoThresholdMv = max(
  (float)PIEZO_MIN_TAP_MV,
  piezoNoiseFloorMv + PIEZO_BASELINE_MARGIN_MV
);

if (piezoArmed) {
  if (pp >= piezoThresholdMv) {
    piezoArmed = false;
    piezoQuietSamples = 0;

    if (now - lastPiezoTapMs >= PIEZO_REFRACTORY_MS) {
      lastPiezoTapMs = now;
      tapCount++;

      Serial.printf(
        "# PIEZO TAP: %.0f mV p-p (baseline %.0f, threshold %.0f) | total taps=%d\n",
        pp,
        piezoNoiseFloorMv,
        piezoThresholdMv,
        tapCount
```

```
      );

      if (!sleepMode) {
        if (!sessionHasTap) {
          firstSessionTapMs = now;
          sessionHasTap = true;
          sessionTapCount = 0;

          Serial.println(
            "# First playing-session tap recorded."
          );
        }

        lastSessionTapMs = now;
        sessionTapCount++;
      }
    }
  } else {
    if (pp < piezoNoiseFloorMv +
          PIEZO_BASELINE_MARGIN_MV) {
      piezoNoiseFloorMv +=
        PIEZO_BASELINE_ALPHA *
        (piezoFilteredMv - piezoNoiseFloorMv);
    }
  }
} else {
  if (pp <= piezoNoiseFloorMv +
```

```
            PIEZO_REARM_MARGIN_MV) {
      piezoQuietSamples++;

      if (piezoQuietSamples >=
          PIEZO_REARM_QUIET_SAMPLES) {
        piezoArmed = true;
        piezoQuietSamples = 0;
      }
    } else {
      piezoQuietSamples = 0;
    }

    if (pp < piezoNoiseFloorMv +
            PIEZO_REARM_MARGIN_MV) {
      piezoNoiseFloorMv +=
        PIEZO_BASELINE_ALPHA *
        (pp - piezoNoiseFloorMv);
    }
  }

  return pp;
}

// Capacitive touch measurement
const float PRESS_RATIO = 1.35;
const float RELEASE_RATIO = 1.20;
```

```
const int PRESS_COUNT = 3;
const int RELEASE_COUNT = 2;

const float MIN_VALID_US = 4.0;

const float CAP_OFFSET_US = 2.3;
const float CAP_US_PER_PF = 0.34;

volatile uint32_t capT1;
volatile bool capGot;

float capBaseline = NAN;
float capLast = NAN;

bool capPressed = false;

unsigned long pressStart = 0;
unsigned long lowStart = 0;

int aboveCount = 0;
int belowCount = 0;

float capPeakTd = 0;

bool touchLatch = false;
bool lastPressedSeen = false;
bool pendingUpload = false;
```

```
bool everTouched = false;
unsigned long lastTouchEnd = 0;

float capPF(float td) {
  float c = (td - CAP_OFFSET_US) / CAP_US_PER_PF;
  return c < 0 ? 0 : c;
}

void IRAM_ATTR onCapEdge() {
  if (!capGot) {
    capT1 = ESP.getCycleCount();
    capGot = true;
  }
}

float capMeasureOnce() {
  REG_WRITE(
    GPIO_OUT_W1TC_REG,
    1UL << PIN_DRIVE
  );

  delayMicroseconds(600);

  capGot = false;

  REG_WRITE(
```

```
    GPIO_OUT_W1TS_REG,
    1UL << PIN_DRIVE
  );

  uint32_t t0 = ESP.getCycleCount();

  delayMicroseconds(600);

  if (!capGot) return NAN;

  return (capT1 - t0) /
         (float)ESP.getCpuFreqMHz();
}

float capMeasure(int n) {
  float v[101];
  int k = 0;

  for (int i = 0; i < n && k < 101; i++) {
    float x = capMeasureOnce();

    if (!isnan(x)) {
      v[k++] = x;
    }
  }

  if (k == 0) return NAN;
```

```
  for (int i = 1; i < k; i++) {
    float x = v[i];
    int j = i - 1;

    while (j >= 0 && v[j] > x) {
      v[j + 1] = v[j];
      j--;
    }

    v[j + 1] = x;
  }

  return v[k / 2];
}

void updateTouch() {
  float td = capMeasure(15);

  if (isnan(td)) return;

  capLast = td;

  if (isnan(capBaseline)) {
    capBaseline = td;
    return;
  }
```

```
if (!capPressed) {
  belowCount = 0;

  if (td > capBaseline * PRESS_RATIO) {
    if (++aboveCount >= PRESS_COUNT) {
      capPressed = true;
      pressStart = millis();
      lowStart = 0;
      aboveCount = 0;
    }
  } else {
    aboveCount = 0;

    if (td < capBaseline * 0.6f) {
      if (lowStart == 0) {
        lowStart = millis();
      } else if (millis() - lowStart > 2000) {
        capBaseline = td;
        lowStart = 0;

        Serial.printf(
          "# baseline was too high -> re-baselined to %.2f us\n",
          td
        );
      }
    } else {
```

```
      lowStart = 0;

      if (td < capBaseline * 1.12f &&
          td > capBaseline * 0.88f) {
        capBaseline +=
          0.005f * (td - capBaseline);
      }
    }
  }
} else {
  aboveCount = 0;

  if (td < capBaseline * RELEASE_RATIO) {
    if (++belowCount >= RELEASE_COUNT) {
      capPressed = false;
      belowCount = 0;

      if (millis() - pressStart < 3000) {
        sleepMode = !sleepMode;

        Serial.printf(
          "# TAP -> %s\n",
          sleepMode ? "SLEEP" : "ACTIVE"
        );
      }
    }
  } else {
```

```
      belowCount = 0;

      if (millis() - pressStart > 30000) {
        capBaseline = td;
        capPressed = false;

        Serial.printf(
          "# long press -> re-baselined to %.2f us\n",
          td
        );
      }
    }
  }
}

// OLED display
unsigned long lastUpload = 0;
const bool LABELED = true;

float sBmpT = 0;
float sShtT = 0;
float sShtH = 0;
float sP = 0;
float sTh = 0;

int nAvg = 0;
```

```
float avgT = NAN;
float avgH = NAN;
float avgP = NAN;

unsigned long lastDisp = 0;
bool modeShown = false;

void updateOLED() {
  if (nAvg > 0) {
    avgT = (sBmpT + sShtT + sTh) /
           (3.0f * nAvg);

    avgH = sShtH / nAvg;
    avgP = sP / nAvg;

    sBmpT = 0;
    sShtT = 0;
    sShtH = 0;
    sP = 0;
    sTh = 0;

    nAvg = 0;
  }

  char l1[32];
  char l2[32];
  char l3[32];
```

```
char l4[32];
char l5[32];

snprintf(
  l1,
  sizeof(l1),
  "%-7s WiFi:%s",
  sleepMode ? "SLEEP" : "ACTIVE",
  WiFi.status() == WL_CONNECTED ? "OK" : "--"
);

snprintf(
  l2,
  sizeof(l2),
  "Temp %.1f C (avg3)",
  avgT
);

snprintf(
  l3,
  sizeof(l3),
  "Humid %.0f %%RH",
  avgH
);

snprintf(
  l4,
```

```
  sizeof(l4),
  "Press %.1f hPa",
  avgP
);

if (isnan(guitarScore)) {
  snprintf(
    l5,
    sizeof(l5),
    "Score --"
  );
} else {
  snprintf(
    l5,
    sizeof(l5),
    "Score %.0f %s%s",
    guitarScore,
    scoreLabel(guitarScore),
    tempDisagree ? "*" : ""
  );
}

u8g2.clearBuffer();
u8g2.setFont(u8g2_font_6x10_tf);

u8g2.drawStr(0, 9, l1);
u8g2.drawStr(0, 21, l2);
```

```
  u8g2.drawStr(0, 33, l3);
  u8g2.drawStr(0, 45, l4);
  u8g2.drawStr(0, 57, l5);

  u8g2.sendBuffer();

  u8g2.setContrast(
    sleepMode ? 10 : 255
  );

  lastDisp = millis();
  modeShown = sleepMode;
}

// Score upload
void uploadGuitarScoreChannel() {
  if (scoreChannelID == 0 ||
      String(scoreWriteAPIKey) ==
      "PASTE_YOUR_SCORE_WRITE_API_KEY") {
    return;
  }

  if (isnan(guitarScore)) return;

  ThingSpeak.setField(1, guitarScore);
  ThingSpeak.setField(2, humScore);
  ThingSpeak.setField(3, tempScore);
```

```
  ThingSpeak.setField(
    4,
    tempDisagree ? 1 : 0
  );

  int response = ThingSpeak.writeFields(
    scoreChannelID,
    scoreWriteAPIKey
  );

  if (response == 200) {
    Serial.printf(
      "# Health Channel OK: Score=%.1f (H=%.0f, T=%.0f, Play=%.0f, Press=%.0f)\n",
      guitarScore,
      humScore,
      tempScore,
      playtimeScore,
      pressureScore
    );
  } else {
    Serial.printf(
      "# Health Channel upload failed. Code: %d\n",
      response
    );
  }
}
```

```
// Environmental upload
void doUpload(bool event) {
  float capOut = event
    ? capPF(capLast)
    : capPF(capPeakTd > 0 ? capPeakTd : capLast);

  int touchedOut = event
    ? (capPressed ? 1 : 0)
    : ((capPressed || touchLatch) ? 1 : 0);

  ThingSpeak.setField(
    1,
    bmp.readTemperature()
  );

  ThingSpeak.setField(
    2,
    bmp.readPressure() / 100.0F
  );

  ThingSpeak.setField(
    3,
    sht31.readTemperature()
  );

  float uploadHumidity = sht31.readHumidity() - 17.5f;
  if (uploadHumidity < 0.0f) {
```

```
  uploadHumidity = 0.0f;
}

ThingSpeak.setField(4, uploadHumidity);
ThingSpeak.setField(5, thermRaw);
ThingSpeak.setField(6, thermKF);
ThingSpeak.setField(7, capOut);
ThingSpeak.setField(8, touchedOut);

int response = ThingSpeak.writeFields(
  channelID,
  writeAPIKey
);

Serial.printf(
  "# ThingSpeak Primary %s (%s): cap=%.1f pF, touched=%d\n",
  response == 200 ? "OK" : "FAILED",
  event ? "event" : "periodic",
  capOut,
  touchedOut
);

uploadGuitarScoreChannel();

lastUpload = millis();
pendingUpload = false;
touchLatch = false;
```

```
  capPeakTd = 0;
}

// Setup
void setup() {
  Serial.begin(115200);

  // Restore saved playtime and pressure baseline.
  preferences.begin("guitar", false);
  cumulativePlayingMs = preferences.getULong64("playMs", 0);
  completedSessions = preferences.getULong("sessions", 0);
  pressureBaselineHpa = preferences.getFloat("pBase", NAN);

  delay(1000);

  Wire.begin(21, 22);
  Wire.setClock(100000);

  analogSetAttenuation(ADC_11db);

  pinMode(PIN_DRIVE, OUTPUT);
  pinMode(PIN_SENSE, INPUT);

  attachInterrupt(
    PIN_SENSE,
    onCapEdge,
    RISING
```

```
);

pinMode(PIN_MUX_SEL, OUTPUT);
digitalWrite(PIN_MUX_SEL, LOW);

// SHT31 initialization
Serial.println("Initializing SHT31...");

if (!sht31.begin(0x44)) {
  Serial.println("SHT31 NOT FOUND at 0x44!");

  while (1) {
    delay(1000);
    Serial.println("Still waiting for SHT31...");
  }
}

float shtTestTemp = sht31.readTemperature();
float shtTestHumidity = sht31.readHumidity();

if (isnan(shtTestTemp) ||
    isnan(shtTestHumidity)) {
  Serial.println(
    "SHT31 initialized, but returned invalid readings."
  );

  while (1) {
```

```
    delay(1000);
    Serial.println(
      "Waiting for valid SHT31 readings..."
    );
  }
}

Serial.printf(
  "SHT31 OK: %.2f C, %.2f %%RH\n",
  shtTestTemp,
  shtTestHumidity
);

// BMP280 initialization
if (!bmp.begin(0x77)) {
  Serial.println("BMP280 NOT FOUND!");

  while (1) {
    delay(1000);
    Serial.println("Still waiting for BMP280...");
  }
}

Serial.println("BMP280 OK.");

// OLED initialization
u8g2.setBusClock(100000);
```

```
u8g2.begin();

u8g2.clearBuffer();
u8g2.setFont(u8g2_font_6x10_tf);

u8g2.drawStr(
  0,
  12,
  "Guitar case monitor"
);

u8g2.drawStr(
  0,
  26,
  "Starting..."
);

u8g2.sendBuffer();

// Wi-Fi
WiFi.mode(WIFI_STA);

Serial.println("Connecting to Wi-Fi...");

WiFi.begin(ssid, password);

int attempts = 0;
```

```
while (WiFi.status() != WL_CONNECTED &&
      attempts < 30) {
  delay(500);
  Serial.print(".");
  attempts++;
}

Serial.println();

if (WiFi.status() == WL_CONNECTED) {
  Serial.println("Wi-Fi connected!");

  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
}

ThingSpeak.begin(client);

u8g2.clearBuffer();

u8g2.drawStr(
  0,
  12,
  "Hands off the"
);
```

```
  u8g2.drawStr(
    0,
    26,
    "touch sensor..."
  );

  u8g2.sendBuffer();

  delay(3000);

  capBaseline = capMeasure(101);
}

// Main loop
void loop() {
  // Environmental measurements
  float bmpTemperature = bmp.readTemperature();

  // Air pressure in hectopascals (hPa)
  float bmpPressure =
    bmp.readPressure() / 100.0F;

  float shtTemperature =
    sht31.readTemperature();

  float rawHumidity =
    sht31.readHumidity();
```

```
// Apply the SHT31 humidity offset.
float shtHumidity =
  rawHumidity - 17.5f;

if (shtHumidity < 0.0f) {
  shtHumidity = 0.0f;
}

// Thermistor measurement
float pin6 = readThermV();

thermRaw =
  (pin6 - THERM_CAL_INTERCEPT) / THERM_CAL_SLOPE;

thermKF = kalman(thermRaw);

// Include cumulative and current-session playtime in the score.
float activeSessionMs = 0.0f;

if (sessionHasTap &&
    lastSessionTapMs >= firstSessionTapMs) {
  activeSessionMs =
    lastSessionTapMs - firstSessionTapMs;
}

float totalPlaytimeMinutes =
```

```
   (cumulativePlayingMs + activeSessionMs) / 60000.0f;

  // Calculate the weighted guitar storage score.
  computeGuitarScore(
    thermKF,
    bmpTemperature,
    shtTemperature,
    shtHumidity,
    bmpPressure,
    totalPlaytimeMinutes
  );

  if (tempDisagree != lastTempDisagree) {
    lastTempDisagree = tempDisagree;

    if (tempDisagree) {
      Serial.printf(
        "# TEMP DISCREPANCY: thermistor=%.2f C, BMP280=%.2f C -> using SHT31 %.2f
C\n",
        thermKF,
        bmpTemperature,
        shtTemperature
      );
    } else {
      Serial.println(
        "# Temperature sensors agree again."
      );
```

```
  }
}

// Piezo measurement
piezoPeakMv = piezoBurst();

// Accumulate display averages
sBmpT += bmpTemperature;
sShtT += shtTemperature;
sShtH += shtHumidity;
sP += bmpPressure;
sTh += thermKF;

nAvg++;

// Serial Monitor output
if (LABELED) {
  Serial.printf(
    "t=%lu ms | BMP280=%.2f C | Pressure=%.2f hPa | therm raw=%.2f C | therm Kalman=%.2f C | cap=%.1f pF | pressed=%s | mode=%s | piezo=%.0f raw / %.0f KF mV p-p (base %.0f / th %.0f) | taps=%d | session taps=%d | RH=%.1f%% | H=%.0f T=%.0f Ptime=%.0f Press=%.0f score=%.1f%s\n",
    millis(),
    bmpTemperature,
    bmpPressure,
    thermRaw,
    thermKF,
    capPF(capLast),
```

```
    capPressed ? "YES" : "no",
    sleepMode ? "SLEEP" : "ACTIVE",
    piezoPeakMv,
    piezoFilteredMv,
    piezoNoiseFloorMv,
    piezoThresholdMv,
    tapCount,
    sessionTapCount,
    shtHumidity,
    humScore,
    tempScore,
    playtimeScore,
    pressureScore,
    guitarScore,
    tempDisagree ? " (temp flag)" : ""
  );
}

unsigned long t = millis();

piezoPeakMv = 0;

while (millis() - t < 250) {
  updateTouch();

  if (!previousSleepMode && sleepMode) {
    finishPlayingSession();
```

```
}

previousSleepMode = sleepMode;

float pp = piezoBurst();

if (pp > piezoPeakMv) {
  piezoPeakMv = pp;
}

if (!isnan(capLast) &&
    capLast > capPeakTd) {
  capPeakTd = capLast;
}

if (capPressed) {
  touchLatch = true;
}

if (capPressed != lastPressedSeen) {
  lastPressedSeen = capPressed;
  pendingUpload = true;

  if (!capPressed) {
    everTouched = true;
    lastTouchEnd = millis();
  }
```

```
  }

  delay(20);
}

// OLED refresh
if (millis() - lastDisp >= DISPLAY_MS ||
    sleepMode != modeShown) {
  updateOLED();
}

// ThingSpeak upload scheduling
if (WiFi.status() == WL_CONNECTED) {
  unsigned long since =
    millis() - lastUpload;

  bool periodicDue =
    since >= (sleepMode ? 120000UL : 20000UL);

  bool eventDue =
    pendingUpload && since >= 15500UL;

  if (periodicDue) {
    doUpload(false);
  } else if (eventDue) {
    doUpload(true);
  }
```

```
    }
}
```